

一.核心流程

1.初始化：

从配置文件读取参数，利用得到的参数创建一个编译器对象，然后执行编译器对象的run方法，首先找到入口文件，进入下一阶段

2.构建（make阶段）

对模块路径和内容解析，生成AST，之后进行AST（抽象语法树）遍历处理，以便分析模块之间的依赖关系和执行逻辑，之后根据模块之间的依赖关系得到一个依赖图（Module Dependency Graph）。

生成的依赖图可以帮助Webpack确定哪些模块可以被拆分成独立的Chunk，并且确定拆分的策略和方式。这也就是webpack**代码分割**的原理。

什么是chunk？

每个Chunk由一个或多个模块组成，它们共同构成了项目的功能和逻辑。在打包完成后，每个Chunk会被编译成一个单独的文件，通常是JavaScript文件，以便在浏览器中加载和执行。

生成chunk可以分为三种情况：

- entry 生成 chunk
- 异步模块生成 chunk
- runtime 生成 chunk

3.生成阶段

合并 chunk，根据 chunk 的不同形成一个或多个 bundle，比如 `main.bundle.js` 是主 bundle，包含了应用程序的主要代码，而一些异步加载的模块则生成不同的 bundle，最终输出到指定的目录，供浏览器加载和执行。

代码分割在构建阶段，生成阶段又组合成一个或多个 bundle.js，那不是相当于没分割嘛？

虽然最终输出的是一个或多个 bundle 文件，但这些 bundle 文件通常是按照逻辑组织的，例如主代码文件、按需加载的代码块等等

代码分割：将应用程序拆分成多个文件可以减少初始加载时间，实现按需加载，并且浏览器可以缓存更改很少的模块

最终输出为一个或多个 bundle.js：将分割的代码片段合并成一个或少数几个 bundle 可以减少 HTTP 请求次数，减少加载时间

二.核心技术

1.sourceMap

当 JavaScript 文件被 webpack 打包后，直接查看的话很难定位到源代码中具体的位置（因为源代码大变样了）。有了 source map，浏览器能够将编译后的代码 **映射** 回源代码，这样在调试时能够在源代码级别上进行错误追踪，提高了开发效率。

（可以在 build 目录下的 static/js 看到，我们除了 chunk.js 以外还会有对应的 chunk.js.map）

2.HMR

模块热替换（Hot Module Replacement, HMR），当某个模块发生变化时，只会重新加载该模块，而不会刷新整个页面，从而提高开发效率。

原理：

因为HMR配置的时候必须要配置dev-server，所以我们下面说的服务器端都是指这个dev-server

1. **Express 启动本地服务：** 开发环境中，一般会使用 Express 工具启动一个本地服务，用于提供静态资源服务和处理动态请求。
2. **服务端和客户端使用 WebSocket 实现长连接：** WebSocket 是一种在客户端和服务端之间建立双向通信的技术。Webpack 使用 WebSocket 在服务端和客户端之间建立长连接，用于实现实时通信。（虽然HTTP也可以实现长连接，但由于HTTP的长连接每次通信时都需要携带一定的头部信息，会降低性能；还有就是HTTP1.1是半双工，websocket可以实现全双工）

半双工

允许数据在两个方向上传输，但是在某一时刻，只允许数据在一个方向上传输，它实际上是一种切换方向的单工通信。

例如：http1.1，对讲机，只能让一个人说一个人听

全双工

允许数据同时在两个方向上传输，因此，全双工通信是两个单工通信方式的结合，它要求发送设备和接收设备都有独立的接收和发送能力。

例如：http2.0、websocket，打电话，双方都能说对方也都能听到

3. **Webpack 监听源文件的变化：** Webpack 监听项目中源文件的变化，比如文件的修改、新增、删除等操作。当开发者保存文件时，Webpack 会检测到文件的变化并触发重新编译。
4. **每次编译生成 hash 值和相应的文件：** 每次重新编译后，Webpack 会生成一个新的 hash 值作为版本标识，并生成对应的模块变化信息的 JSON 文件以及更新后的模块代码的 JS 文件。

5. **通过 Socket 向客户端推送编译的 hash 戳：** 编译完成后，Webpack 会通过已建立的 WebSocket 连接向客户端推送当前编译的 hash 值。
6. **客户端监听到文件改动推送的 hash 戳并进行比对：** 客户端的 WebSocket 监听到服务端推送过来的 hash 值后，会将其与上一次记录的 hash 值进行比较。
7. **根据 hash 值是否一致决定是否更新：** 如果当前推送的 hash 值与上一次不一致，则表示有文件发生了变化，客户端会通过 AJAX 或 JSONP 向服务端获取最新的资源。如果一致，则表示文件未发生变化，客户端可以继续使用缓存。
8. **使用内存文件系统进行局部刷新：** 在获取到最新资源后，客户端会使用内存文件系统将发生变化的模块代码进行局部更新，从而实现页面的局部刷新，而不是重新加载整个页面。3

3.oneOf

通常在处理特定类型文件时，可能有多个 loader 或者插件可供选择，匹配到一个 loader 后，后面的就不会再继续匹配了。不需要每个文件把所有的 loader 都询问一遍，可以提高 loader 的执行效率

4.include和exclude

我们希望 webpack 不需要处理 node_modules 下的 js 文件，可以进行优化

这两个配置项其实是和 test 同类型的配置，只不过含义不同，优先级也不同

exclude 优先级最高，当 include 和 exclude 同时存在时，以 exclude 的为主。也就是说如果 exclude 和 include 冲突了，include 无效

并且 include 一般不能和 exclude 同时存在，除非 exclude 是 include 的子集

test 指项目中的所有文件，include 指定需要处理文件，exclude 指定除 value 之外都处理

value 值和 test 语法都一样，可以是正则表达式也可以是一个绝对路径

5.eslint

ESLint是一个JavaScript代码检查工具，用于识别和报告代码中的模式问题，同时也可以自己配置规范，以此使得项目代码规范统一

6.cache

Webpack的缓存（Cache）是指在构建过程中使用一些机制来存储中间结果，以便在后续构建中可以复用这些结果，从而提高构建效率。一般我们对js文件和eslint下配置缓存：

```
test: /\. (js|mjs|jsx|ts|tsx)$/,
  include: paths.appSrc,
  loader: require.resolve('babel-loader'),
  options: {
    customize: require.resolve(
      'babel-preset-react-app/webpack-
overrides'
    ),
    presets: [
      [
        require.resolve('babel-preset-react-
app'),
        {
          runtime: hasJsxRuntime ? 'automatic'
: 'classic',
        },
      ],
    ],
    // @remove-on-eject-begin
    babelrc: false,
    configFile: false,
    plugins: [
      isEnvDevelopment &&
```

```
        shouldUseReactRefresh &&
        require.resolve('react-refresh/babel'),
    ].filter(Boolean),

    cacheDirectory: true,
    // See #6846 for context on why
    cacheCompression is disabled
    cacheCompression: false,
    compact: isEnvProduction, /// 缓存配置
},
```

原理

Webpack缓存分为内存缓存和持久缓存：

- 内存缓存（默认）
 - 内存缓存本质是由 `Map` 内存存储数据（`key` 存储索引，`value` 存储缓存内容），可分为普通缓存和带GC策略缓存。放入memory中，读取更加快速，但是容量小
 - 普通缓存：由 `Map` 实现存储，不考虑缓存淘汰；
 - 带GC策略缓存：使用 `cache`、`oldCache` 实现LRU缓存淘汰算法；（简单来讲就是最近一直没有使用的缓存算法清除）
- 持久缓存
 - 相比于内存缓存，持久缓存设计上更加复杂，从架构设计上可以将其分为插件接入层、缓存策略层以及、序列化层。并且放入磁盘中，虽然读取速度没有内存缓存快，但是容量大而且持久性强
 - 插件接入层：提供缓存策略接入编译器的能力；
 - 缓存策略层：提供缓存管理以及缓存有效性检测的能力；
 - 序列化层：提供缓存对象与磁盘文件的存储、还原的能力

7.多进程打包（与loader相关知识关联性很大）

因为thread-loader多线程开启过程需要耗费时间大概为600ms，多线程之间的通信也会消耗时间，因此如果项目规模不大那么不需要使用，项目规模大的时候这个优化才有明显提升

- **启动时拦截 Loader 执行链：**

Thread-loader 在启动时以 **pitch 方式拦截** Loader 执行链。这意味着它会在 Loader 执行之前介入，并有机会修改 Loader 的执行顺序或者传递参数给后续 Loader。（可是我们把这个threar-loader放在use数组的开头，那么它又是如何以所谓pitch方式拦截的呢？）

- **分析 Webpack 配置对象：**

Thread-loader 分析 Webpack 配置对象，特别是获取 thread-loader 后面的 Loader 列表。这些 Loader 列表将在后续的转译过程中被传递给工作子进程。

- **创建工作子进程：**

Thread-loader 使用 `child_process.spawn` 创建工作子进程。在这个子进程中，会执行实际的 Loader 转译工作。

- **调用 loader-runner 转译文件内容：**

在子进程中，调用 `loader-runner` 来执行 Loader 相关的转译逻辑。这意味着 Thread-loader 复用了 `loader-runner` 的功能，从而达到与普通 Loader 相同的效果。

- **将转译结果传回主进程：**

转译完成后，子进程将转译结果传回给主进程。主进程可以继续处理这些结果，例如将它们传递给下一个 Loader 或者直接输出到最终的输出文件中。

8.treeShaking

Webpack 的 Tree Shaking 是指通过静态分析代码的依赖关系，识别并删除未使用的代码，以减少打包后的文件体积。Webpack 的 Tree Shaking 是指通过静态分析代码的依赖关系，识别并删除未使用的代码，以减少打包后的文件体积。webpack默认开启，无需开发者配置

原理：

Webpack 中，Tree-shaking 的实现一是先「标记」出模块导出值中哪些没有被用过，二是使用 Terser 删掉这些没被用到的导出语句。标记过程大致可划分为三个步骤：

- Make 阶段，收集模块导出变量并记录到模块依赖关系图 ModuleGraph 变量中
- Seal 阶段，遍历 ModuleGraph 标记模块导出变量有没有被使用 (usedExports)
- 生成产物时，若变量没有被其它模块使用则删除对应的导出语句 (terser)

“标记功能需要配置 `optimization.usedExports = true` 开启”

9. 图片压缩和css文件压缩

```
optimization: {
  // 压缩的操作
  minimizer: [
    // 压缩css
    new CssMinimizerPlugin(),
    // 压缩js
    new TerserWebpackPlugin({
      parallel: threads, // 开启多进程和设置进程数量
    }),
    // 压缩图片
    new ImageMinimizerPlugin({
```

10. codeSplit

Code Splitting，即代码分割，是一种将代码拆分成多个小块的技术。这些小块可以独立加载和执行，从而实现按需加载，提高应用的性能。Webpack通过其内置的插件和加载器，支持多种方式的代码分割。

在构建阶段我们生成了那个依赖图（Module Dependency Graph），根据这个生成可以分割的 chunk

除此之外，Webpack 会在满足下面的条件时自动进行 chunk 的分割

- 新的 chunk 可以被共享或者 chunk 里的 module 来自于 node_modules 文件夹
- 新的 chunk 文件大小大于 30kb(压缩前)
- 按需加载的 chunk 内部的最大并行请求数不超过 5 个
- 初始化页面的并行请求的最大数量不超过 3 个

当然我们可以修改配置文件来更改分割的条件

11.preload和prefetch

我们前面已经做了代码分割，同时会使用 import 动态导入语法来进行代码按需加载（我们也叫做加载，比如路由懒加载就是这样实现的）。

但是加载速度还不够好，比如：是用户点击按钮时才加载这个资源的，如果资源体积很大，那么用户会感觉到明显卡顿效果。

我们想在浏览器空闲时间，加载后续需要使用的资源。我们就需要用上 Preload 或 Prefetch 技术。

是什么

- Preload：告诉浏览器立即加载资源。
- Prefetch：告诉浏览器在空闲时才开始加载资源。

它们共同点：

- 都只会加载资源，并不执行。
- 都有缓存。

它们区别：

- Preload 加载优先级高，Prefetch 加载优先级低。
- Preload 只能加载当前页面需要使用的资源，Prefetch 可以加载当前页面资源，也可以加载下一个页面需要使用的资源。

12.文件命名用哈希值（runtimeChunk）

Webpack 在构建过程中生成的文件名通常是由 webpack 根据一些算法生成的哈希值来命名的，而不是直接使用原始的文件名。这样做的目的是为了

防止缓存问题和提高浏览器的并行加载性能。

这个奇怪的字符串（如 `933abf.chunk.js`）通常是根据文件内容计算出的哈希值，Webpack 使用文件内容的哈希值作为文件名的一部分，这样如果文件内容发生变化，哈希值也会变化，从而可以保证生成的文件名是唯一的，并且可以很容易地识别出是否发生了变化。

这种做法有以下的好处是**防止缓存问题**：当文件内容发生变化时，文件名也会发生变化，浏览器会重新请求新的文件，从而避免了缓存问题。

但是这样带来了新的问题，如果一个文件A依赖文件B，文件B内容发生变化，文件名也会发生变化。A依赖B，那么A的文件内容也相当于发生了变化，因此A的文件名也会改变。但是这样会使得A和B的缓存都失效，造成性能的损失。我们的方法是创建一个runtime文件，里面只保存对于文件的引用。当引用的文件内容改变，它的文件名以及对饮runtime文件的文件名都会改变，但是引用文件的那个文件文件名不会改变，缓存依然有效。

方法：`runtimeChunk`

```
splitChunks: {
  chunks: "all",
  // 其他都用默认值
},
runtimeChunk: {
  name: (entrypoint) => `runtime~${entrypoint.name}.js`,
},
},
```

13.PWA

PWA 全称 Progress Web Application，也就是渐进式web应用，是由 Google 在2015年就提出的概念。它本质上是一个普通的 web 页面，但是由于使用了 Service Worker、Notification/Push API、Manifest 等新的技术，使得这个web页面能够拥有媲美原生APP的用户体验，比如离线访问、消息推送等

在webpack中使用workbox-webpack-plugin引入然后配置这个插件

14.runtime

在运行环境中，webpack对模块进行解析，加载的代码，这些代码是打包过程中webpack引入的代码片段，会被注入到最终的打包文件中。

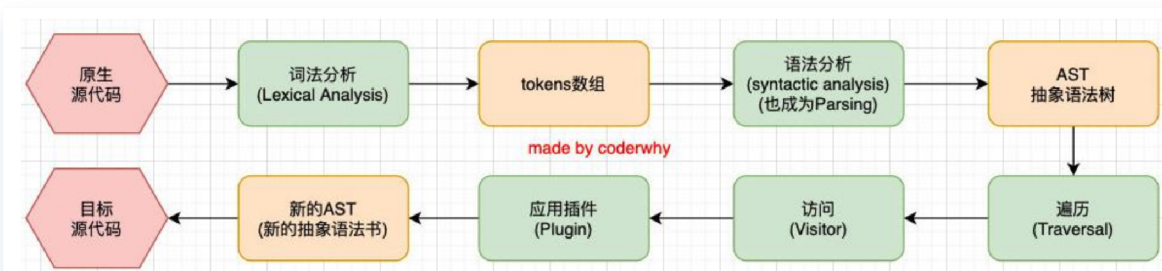
15.Babel

■ babel是如何做到将我们的一段代码（ES6、TypeScript、React）转成另外一段代码（ES5）的呢？

- 从一种源代码（原生语言）转换成另一种源代码（目标语言），这是什么的工作呢？
- 就是**编译器**，事实上我们可以将babel看成就是一个编译器。
- Babel编译器的作用就是将我们的源代码，转换成浏览器可以直接识别的另外一段源代码；

■ Babel也拥有编译器的工作流程：

- 解析阶段（Parsing）
- 转换阶段（Transformation）
- 生成阶段（Code Generation）



babel-preset

- 如果我们一个个去安装使用插件，那么需要手动来管理大量的babel插件，我们可以直接给webpack提供一个preset，webpack会根据我们的预设来加载对应的插件列表，并且将其传递给babel。

16.BrowserList

通过在项目配置文件中指定BrowserList，开发者可以告诉工程化工具哪些浏览器需要支持，以及支持的最低版本。然后，工具会根据这些信息来自动进行代码转译、压缩、打包等操作，以确保最终生成的代码能够在指定的目标浏览器中正确运行。

■ 我们如何可以配置browserslist呢？两种方案：

- 方案一：在package.json中配置；
- 方案二：单独的一个配置文件.browserslistrc文件；

■ 方案一：package.json配置：

```
"browserslist": [
  "last 2 version",
  "not dead",
  "> 0.2%"
]
```

■ 方案二：.browserslistrc文件

```
webpack打包css资源 > .browserslistrc
1  > 0.5%
```

1. 然后，在Babel的配置文件（通常是 `.babelrc` 或 `babel.config.js`）中，确保使用 `@babel/preset-env` 预设，并启用 `useBuiltIns` 选项，将其设置为 `usage`。这样Babel会根据BrowserList的配置自动添加所需的polyfill，以确保代码在目标浏览器中正确运行。例如：

```

{
  "presets": [
    [
      "@babel/preset-env",
      {
        "useBuiltIns": "usage",
        "corejs": "3.0.0" // 设置所需的core-js版本
      }
    ]
  ]
}

```

■ 什么时候会用到polyfill呢?

- 比如我们使用了一些语法特性（例如：Promise, Generator, Symbol等以及实例方法例如Array.prototype.includes等）
- 但是某些浏览器压根不认识这些特性，必然会报错；
- 我们可以使用polyfill来填充或者说打一个补丁，那么就会包含该特性了；

上面那个corejs就是一polyfill

webpack中配置babel:

```

const path = require('path');

module.exports = {
  entry: './src/index.js', // 入口文件
  output: {
    path: path.resolve(__dirname, 'dist'), // 输出目录
    filename: 'bundle.js' // 输出文件名
  },
  module: {
    rules: [
      {
        test: /\.js$/, // 匹配所有.js文件

```

```

    exclude: /node_modules/, // 排除node_modules目录
    use: {
      loader: 'babel-loader', // 使用babel-loader处理
      options: {
        presets: ['@babel/preset-env'] // 使用
        @babel/preset-env进行转译
      }
    }
  }
}
];
};

```

补充：webpack配置devserver---proxy代理



Proxy代理

■ proxy是我们开发中非常常用的一个配置选项，它的目的设置代理来解决跨域访问的问题：

- 比如我们的一个api请求是 http://localhost:8888，但是本地启动服务器的域名是 http://localhost:8080，这个时候发送网络请求就会出现跨域的问题；
- 那么我们可以将请求先发送到一个代理服务器，代理服务器和API服务器没有跨域的问题，就可以解决我们的跨域问题了；

■ 我们可以进行如下的设置：

- target: 表示的是代理到的目标地址，比如 /api-hy/moment会被代理到 http://localhost:8888/api-hy/moment；
- pathRewrite: 默认情况下，我们的 /api-hy 也会被写入到URL中，如果希望删除，可以使用pathRewrite；
- changeOrigin: 它表示是否更新代理后请求的headers中host地址；



historyApiFallback

- **historyApiFallback**是开发中一个非常常见的属性，它主要的作用是解决SPA页面在路由跳转之后，进行页面刷新时，返回404的错误。
- **boolean值**：默认是false
 - 如果设置为true，那么在刷新时，返回404错误时，会自动返回 index.html 的内容；
- **object类型的值**，可以配置rewrites属性：
 - 可以配置from来匹配路径，决定要跳转到哪一个页面；

17.terser

Webpack 通常使用 **Terser** 来压缩 JavaScript 文件。Terser 是一个用于 JavaScript 的 JavaScript 压缩器/美化器工具，能够帮助减小 JavaScript 文件的大小，从而加快网页加载速度。同时在treeshaking中也有一些作用



Terser在webpack中配置

- 真实开发中，我们不需要手动的通过terser来处理我们的代码，我们可以直接通过webpack来处理：
 - 在webpack中有一个minimizer属性，在production模式下，默认就是使用TerserPlugin来处理我们的代码的；
 - 如果我们对默认的配置不满意，也可以自己来创建TerserPlugin的实例，并且覆盖相关的配置；
- 首先，我们需要打开minimize，让其对我们的代码进行压缩（默认production模式下已经打开了）
- 其次，我们可以在minimizer创建一个TerserPlugin：
 - **extractComments**：默认值为true，表示会将注释抽取到一个单独的文件中；
 - ✓ 在开发中，我们不希望保留这个注释时，可以设置为false；
 - **parallel**：使用多进程并发运行提高构建的速度，默认值是true
 - ✓ 并发运行的默认数量：os.cpus().length - 1；

```
optimization: {  
  minimize: true,  
  minimizer: [  
    new TerserPlugin({  
      extractComments: false,  
      terserOptions: {  
        compress: {  
          unused: false  
        }  
      }  
    })  
  ]  
}
```

18.Scope Hoisting

■ 什么是Scope Hoisting呢？

- Scope Hoisting从webpack3开始增加的一个新功能；
- 功能是对作用域进行提升，并且让webpack打包后的代码更小、运行更快；

■ 默认情况下webpack打包会有很多的函数作用域，包括一些（比如最外层的）IIFE：

- 无论是从最开始的代码运行，还是加载一个模块，都需要执行一系列的函数；
- Scope Hoisting可以将函数合并到一个模块中来运行；

■ 使用Scope Hoisting非常的简单，webpack已经内置了对应的模块：

- 在production模式下，默认这个模块就会启用；
- 在development模式下，我们需要自己来打开该模块；

```
new webpack.optimize.ModuleConcatenationPlugin(),
```

（IIFE是立即执行函数表达式（Immediately Invoked Function Expression）的缩写）

补充：Object.freeze()

`Object.freeze()` 是 JavaScript 中的一个方法，它用于冻结一个对象，使其不能添加新属性，不能修改已有属性的值，也不能删除已有属性。这个方法在 ECMAScript 5 中被引入。

当你调用 `Object.freeze(obj)` 时，它会返回一个冻结后的对象 `obj`。冻结后的对象不能被修改，任何试图修改其属性的操作都会被忽略，严格模式下还会抛出错误。

下面是一个简单的示例说明了 `Object.freeze()` 的用法：

```
const obj = {  
  prop1: 123,  
  prop2: 'hello'  
};  
  
Object.freeze(obj);
```

```
// 尝试修改已有属性
obj.prop1 = 456; // 这行代码无效，属性值不会被修改
console.log(obj.prop1); // 输出：123

// 尝试添加新属性
obj.prop3 = 'world'; // 这行代码无效，新属性不会被添加
console.log(obj.prop3); // 输出：undefined

// 尝试删除属性
delete obj.prop2; // 这行代码无效，属性不会被删除
console.log(obj.prop2); // 输出：'hello'

// 严格模式下会抛出错误
'use strict';
obj.prop1 = 789; // TypeError: Cannot assign to read only
property 'prop1' of object '#<Object>'
```

`Object.freeze()` 是一种保护对象不被意外修改的方式，但需要注意的是它只能冻结对象的第一层属性，对于嵌套对象，嵌套对象的属性不会被冻结，仍然可以被修改。

三.再深入源码

在Webpack配置中，有些地方可以使用相对路径，而有些地方则必须使用绝对路径。这是因为Webpack在不同的地方对路径解析的方式有所不同，需要使用不同的路径格式来满足其要求。

可以使用相对路径的地方：

1. **入口配置**：在Webpack的入口配置中，通常可以使用相对路径来指定入口文件的路径。例如：

```
module.exports = {  
  entry: './src/index.js',  
};
```

2. loader配置：在Webpack的loader配置中，可以使用相对路径来指定loader的路径。例如：

```
module.exports = {  
  module: {  
    rules: [  
      {  
        test: /\.js$/,  
        use: 'babel-loader',  
        include: path.resolve(__dirname, 'src'),  
      },  
    ],  
  },  
};
```

必须使用绝对路径的地方：

1. output配置：在Webpack的output配置中，必须使用绝对路径来指定输出文件的路径。例如：

```
module.exports = {  
  output: {  
    path: path.resolve(__dirname, 'dist'),  
    filename: 'bundle.js',  
  },  
};
```

2. 其他绝对路径要求的地方：有些插件或者功能可能要求使用绝对路径，这取决于具体的配置和使用情况。

总的来说，Webpack在某些地方要求使用绝对路径，主要是为了确保路径的准确性和一致性，以避免出现意外的错误或者不可预料的行为。因此，在编写Webpack配置时，建议根据具体的要求来选择使用相对路径或者绝对路径。

loader和plugin的区别

Webpack中的Loader和Plugin是两个不同的概念，它们分别用于处理不同的任务，并且在Webpack的构建过程中扮演着不同的角色。

1. Loader：

- **作用：**Loader用于对模块的源代码进行转换。在Webpack的构建过程中，当遇到不同类型的文件（如.js、.css、.png等）时，Webpack会使用相应的Loader对文件进行处理，**将其转换为模块**，并添加到依赖图中。
- **使用方式：**Loader通过module.rules配置项来使用。每个Loader通常都包含在一个数组中，按照顺序执行。Loader可以链式调用，每个Loader都对文件进行一次转换。
- **示例：**常见的Loader有babel-loader（用于将ES6+代码转换为ES5代码）、css-loader（用于解析.css文件）、file-loader（用于处理文件资源）等。

2. Plugin：

- **作用：**Plugin用于扩展Webpack的功能。通过在Webpack构建过程的不同阶段挂载钩子函数，Plugin可以执行各种任务，如优化资源、压缩代码、添加环境变量等。
- **使用方式：**Plugin通过配置webpack.config.js中的plugins选项来使用。每个Plugin是一个独立的实例，可以在Webpack构建过程中调用不同的钩子函数，执行相应的任务。

- **示例**：常见的Plugin有HtmlWebpackPlugin（用于自动生成HTML文件）、MiniCssExtractPlugin（用于提取CSS文件）、UglifyJsPlugin（用于压缩JS代码）等。

1.loader

webpack 只能直接处理 javascript 格式的代码。任何非 js 文件都必须被预先处理转换为 js 代码，才可以参与打包。loader（加载器）就是这样一个代码转换器。它由 webpack 的 `loader runner` 执行调用，接收原始资源数据作为参数（当多个加载器联合使用时，上一个loader的结果会传入下一个loader），最终输出 javascript 代码（和可选的 source map）给 webpack 做进一步编译。

Loader 主要工作于 Webpack 的编译阶段（Compile），在这个阶段，Webpack 会对每个模块应用指定的 Loader 进行转换处理，然后再将转换后的模块进行后续的构建和打包操作。

Loader 执行顺序

1. 分类

- **前置 loader**（pre-loader）：在正常加载器之前运行的加载器。前置加载器用于在资源被正常加载器处理之前，对资源进行一些预处理或其他操作。例如，在加载 JavaScript 文件之前，可能会使用前置加载器进行代码静态分析或语法检查。
- **普通 loader**（normal loader）：这是最常见的加载器类型，用于正常加载和转换资源。大多数资源文件都会通过普通加载器进行处理，例如将 Sass 文件转换为 CSS、将 TypeScript 文件转换为 JavaScript 等。
- **内联 loader**（inline loader）：内联加载器是直接在代码中引用的加载器，而不是通过配置文件指定。它们通常用于临时调试或针对特定代码段的加载器。在代码中使用内联加载器可以方便地对特定代码片段进行单独的加载器处理，而无需修改配置文件。
- **后置 loader**（post-loader）：在普通加载器完成其工作后运行的加载器。后置加载器用于在资源加载和转换完成后，对输出进行进一步处

理。例如，可能会使用后置加载器来优化和压缩生成的 JavaScript 或 CSS 文件。

2. 执行优先级

- 4类 loader 的执行优先级为： `pre > normal > inline > post` 。
- 相同优先级的 loader 执行顺序为： `从右到左，从下到上` 。

3. 前缀的作用

内联 loader 可以通过添加不同前缀，跳过其他类型 loader。

- `!` 跳过 normal loader。
- `-!` 跳过 pre 和 normal loader。
- `!!` 跳过 pre、normal 和 post loader。

这些前缀在很多场景下非常有用。

如何开发一个loader

loader 是一个导出一个函数的 node 模块。

最简单的 loader

当只有一个 loader 应用于资源文件时，它接收源码作为参数，输出转换后的 js 代码。

```
// loaders/simple-loader.js
module.exports = function loader (source) {
  console.log('simple-loader is working');
  return source;
}
```

这就是一个最简单的 loader 了，这个 loader 啥也没干，就是接收源码，然后原样返回

带 pitch 的 loader

`pitch` 是 loader 上的一个方法，它的作用是阻断 loader 链。

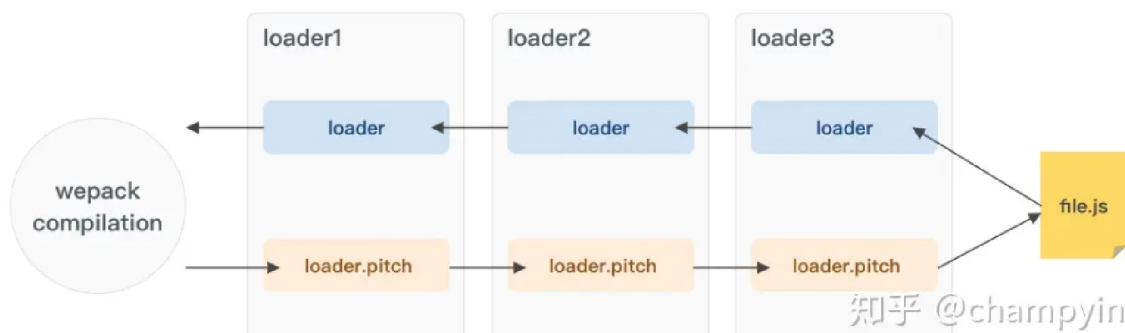
```
// loaders/simple-loader-with-pitch.js
module.exports = function (source) {
  console.log('normal excution');
  return source;
}

// loader上的pitch方法, 非必须
module.exports.pitch = function() {
  console.log('pitching graph');
  // todo
}
```

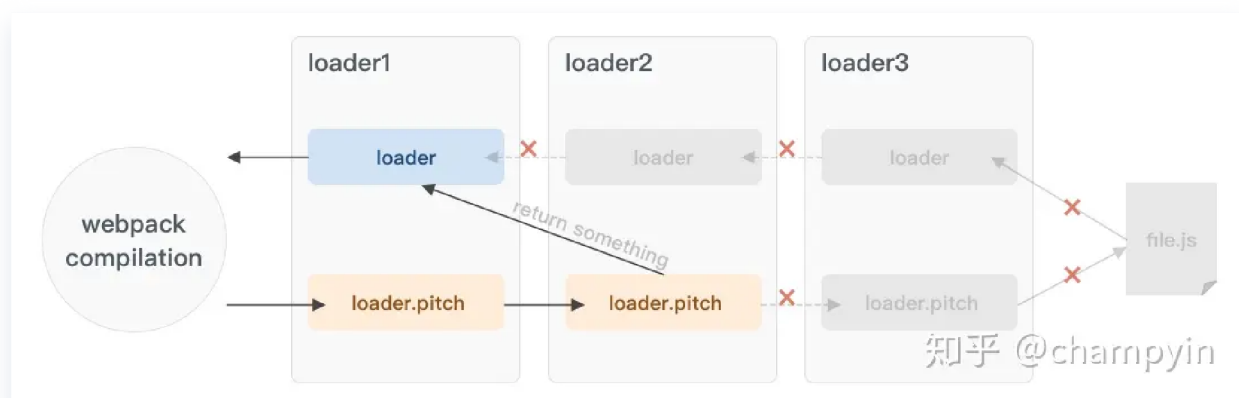
`pitch` 方法不是必须的。如果有 `pitch`，loader 的执行则会分为两个阶段：`pitch` 阶段 和 `normal execution` 阶段。`webpack` 会先从左到右执行 loader 链中的每个 loader 上的 `pitch` 方法（如果有），然后再从右到左执行 loader 链中的每个 loader 上的普通 loader 方法。

```
use: ['loader1', 'loader2', 'loader3']
```

真实的 loader 执行过程是：



在这个过程中如果任何 pitch 有返回值，则 loader 链被阻断。webpack 会跳过后面的所有 pitch 和 loader，直接进入上一个 loader 的 `normal execution`。



pitch 方法有3个参数：

1. remainingRequest: loader链中排在自己后面的 loader 以及资源文件的绝对路径以 `!` 作为连接符组成的字符串。
2. precedingRequest: loader链中排在自己前面的 loader 的绝对路径以 `!` 作为连接符组成的字符串。
3. data: 每个 loader 中存放在上下文中的固定字段，可用于 pitch 给 loader 传递数据。

2.plugin

tapable

webpack 就像一条生产线，要经过一系列处理流程后才能将源文件转换成输出结果。这条生产线上的每个处理流程的职责都是单一的，多个流程之间有存在依赖关系，只有完成当前处理后才能交给下一个流程去处理。

插件就像是一个插入到生产线中的一个功能，在特定的时机对生产线上的资源做处理。webpack 通过 Tapable 来组织这条复杂的生产线。webpack 在编译代码过程中，会触发一系列 Tapable 对象上的钩子事件，插件所做

的，就是往钩子上面注册事件，然后执行回调函数。这样，当 webpack 构建的时候，插件注册的事件就会随着钩子的触发而执行了。

webpack 里的几个非常重要的对象，`Compiler`，`Compilation` 和 `JavascriptParser` 都继承了 `Tapable` 类，它们身上挂着丰富的钩子。

- **Compiler Hooks**

Compiler 编译器模块是创建编译实例的主引擎。大多数面向用户的插件都首先在 Compiler 上注册。

compiler上暴露的一些常用的钩子：

钩子	类型	什么时候调用
<code>run</code>	<code>AsyncSeriesHook</code>	在编译器开始读取记录前执行
<code>compile</code>	<code>SyncHook</code>	在一个新的 <code>compilation</code> 创建之前执行
<code>compilation</code>	<code>SyncHook</code>	在一次 <code>compilation</code> 创建后执行插件
<code>make</code>	<code>AsyncParallelHook</code>	完成一次编译之前执行
<code>emit</code>	<code>AsyncSeriesHook</code>	在生成文件到 <code>output</code> 目录之前执行，回调参数： <code>'compilation'</code>
<code>afterEmit</code>	<code>AsyncSeriesHook</code>	在生成文件到 <code>output</code> 目录之后执行
<code>assetEmitted</code>	<code>AsyncSeriesHook</code>	生成文件的时候执行，提供访问产出文件信息的入口，回调参数： <code>'file'</code> ， <code>'info'</code>
<code>done</code>	<code>AsyncSeriesHook</code>	一次编译完成后执行，回调参数： <code>'stats'</code>

- **Compilation Hooks**

Compilation 是 **Compiler** 用来创建一次新的编译过程的模块。一个 **Compilation** 实例可以访问所有模块和它们的依赖。在一次编译阶段，模块被加载、封装、优化、分块、散列和还原。

Compilation 也继承了 Tapable 并提供了很多生命周期钩子。

Compilation 上暴露的一些常用的钩子：

钩子	类型	什么时候调用
<i>buildModule</i>	<i>SyncHook</i>	在模块开始编译之前触发，可以用于修改模块
<i>succeedModule</i>	<i>SyncHook</i>	当一个模块被成功编译，会执行这个钩子
<i>finishModules</i>	<i>AsyncSeriesHook</i>	当所有模块都编译成功后被调用
<i>seal</i>	<i>SyncHook</i>	当一次 <i>compilation</i> 停止接收新模块时触发
<i>optimize</i>	<i>SyncHook</i>	在优化阶段的开始执行
<i>optimizeAssets</i>	<i>AsyncSeriesHook</i>	优化所有存放在' <i>compilation.assets</i> '的所有资源。回调参数： <i>'assets'</i>

• JavascriptParser Hooks

Parser 解析器实例在 Compiler 编译器中产生，用于解析 webpack 正在处理的每个模块。我们可以用它提供的 Tapable 钩子自定义解析过程。

JavascriptParser 上暴露的一些常用的钩子：

钩子	类型	什么时候调用
<i>evaluate</i>	<i>SyncBailHook</i>	在计算表达式的时候调用。
<i>statement</i>	<i>SyncBailHook</i>	为代码片段中每个已解析的语句调用的通用钩子
<i>import</i>	<i>SyncBailHook</i>	为代码片段中每个 <i>import</i> 语句调用，回调参数： <i>'statement'</i> ， <i>'source'</i>
<i>export</i>	<i>SyncBailHook</i>	为代码片段中每个 <i>export</i> 语句调用，回调参数： <i>'statement'</i>
<i>call</i>	<i>SyncBailHook</i>	解析一个 <i>call</i> 方法的时候调用，回调参数： <i>'expression'</i>
<i>program</i>	<i>SyncBailHook</i>	解析一个表达式的时候调用，回调参数： <i>'expression'</i>

Tapable 对象统一暴露了三个方法给插件，用于注入不同类型的自定义构建行为：（比如Tapable.randomHooks.tap）

- **tap**：注册钩子，同步钩子和异步钩子都可以使用。
- **tapAsync**：回调方式注册异步钩子。
- **tapPromise**：Promise方式注册异步钩子。

这几个方法都遵循发布-订阅模式（Publish-Subscribe Pattern），即监听事件--执行回调，监听的事件是插件的名字

补充-----钩子类型：

1. **SyncHook**：同步钩子，用于在特定事件触发时同步执行注册的回调函数。

2. `SyncBailHook` : 同步保险钩子, 与 `SyncHook` 类似, 但是如果某个回调函数返回非 `undefined` 的值, 则停止后续回调的执行。
3. `SyncWaterfallHook` : 同步瀑布钩子, 与 `SyncHook` 类似, 但是每个回调函数的返回值将作为下一个回调函数的参数。
4. `SyncLoopHook` : 同步循环钩子, 允许在某个回调函数执行完毕后, 重新触发相同的事件。
5. `AsyncParallelHook` : 异步并行钩子, 用于在特定事件触发时异步并行执行注册的回调函数。
6. `AsyncParallelBailHook` : 异步并行保险钩子, 与 `AsyncParallelHook` 类似, 但是如果某个回调函数返回非 `undefined` 的值, 则停止后续回调的执行。
7. `AsyncSeriesHook` : 异步串行钩子, 用于在特定事件触发时异步串行执行注册的回调函数。
8. `AsyncSeriesBailHook` : 异步串行保险钩子, 与 `AsyncSeriesHook` 类似, 但是如果某个回调函数返回非 `undefined` 的值, 则停止后续回调的执行。
9. `AsyncSeriesLoopHook` : 异步串行循环钩子, 与 `AsyncSeriesHook` 类似, 但是允许在某个回调函数执行完毕后, 重新触发相同的事件。
10. `AsyncSeriesWaterfallHook` : 异步串行瀑布钩子, 与 `AsyncSeriesHook` 类似, 但是每个回调函数的返回值将作为下一个回调函数的参数。

compiler与compilation

webpack的工作流程:

- 将命令行参数与 `webpack 配置文件` 合并、解析得到参数对象。
- 参数对象传给 webpack 执行得到 `Compiler` 对象。
- 执行 `Compiler` 的 `run` 方法开始编译。每次执行 `run` 编译都会生成一个 `Compilation` 对象。
- 触发 `Compiler` 的 `make` 方法分析入口文件, 调用 `compilation` 的 `buildModule` 方法创建主模块对象。
- 生成入口文件的 `AST(抽象语法树)`, 以便分析模块之间的依赖关系和执行逻辑, 之后根据模块之间的依赖关系得到一个依赖图 (Module

Dependency Graph)。生成的依赖图可以帮助Webpack确定哪些模块可以被拆分成独立的 Chunk

- 执行 `compilation` 的 `seal` 方法对每个 `chunk` 进行整理、优化、封装。
- 最后执行 `Compiler` 的 `emitAssets` 方法把生成的文件输出到 `output` 的目录中。

compiler是webpack的入口执行实例，其主要功能是监控代码、启动和终止一个compilation等宏观任务。compilation是具体负责核心编译工作的，主要是模块依赖分析，优化、连接，打包等工作。打个粗略的比方，compiler是人，compilation是人的大脑，架构设计上是一致的。

自定义一个plugin

一个 webpack plugin 由如下部分组成：

1. 一个命名的 Javascript 方法或者 JavaScript 类。
2. 它的原型上需要定义一个叫做 `apply` 的方法。
3. 注册一个事件钩子。
4. 操作webpack内部实例特定数据。
5. 功能完成后，调用webpack提供的回调

一个基本的 plugin 代码结构大致长这个样子：

```
// plugins/MyPlugin.js
```

```
class MyPlugin {  
  apply(compiler) {  
    compiler.hooks.done.tap('My Plugin', (stats) => {  
      console.log('Bravo!');  
    });  
  }  
}  
  
module.exports = MyPlugin;
```

补充：什么是PWA

■ 整个目录结构都非常好理解，只是有一个PWA相关的概念：

- PWA全称**Progressive Web App**，即**渐进式WEB应用**；
- 一个 PWA 应用首先是一个网页，可以通过 Web 技术编写出一个网页应用；
- 随后添加上 **App Manifest** 和 **Service Worker** 来实现 PWA 的**安装和离线**等功能；
- 这种Web存在的形式，我们也称之为是 **Web App**；

■ PWA解决了哪些问题呢？

- 可以**添加至主屏幕**，点击主屏幕图标可以实现启动动画以及隐藏地址栏；
- 实现**离线缓存功能**，即使用户手机没有网络，依然可以使用一些离线功能；
- 实现了**消息推送**；
- 等等一系列类似于Native App相关的功能；

■ 更多PWA相关的知识，可以自行去学习更多；

- https://developer.mozilla.org/zh-CN/docs/Web/Progressive_web_apps